Smart Bookstore Management System

MERN Stack

Internship Report

Submitted by

SHUBHANJALI

University	Indian Institute of Information Technology Allahabad
Branch	Information Technology
Company	The Skybrisk — Internship Program
Mentor	Mr. Govind Bhokare
Project	Month 2 — Bookstore Management System (REST API + React)
Tech Stack	MongoDB, Express.js, React.js, Node.js
Duration	March – April 2026
Year	2026

Acknowledgement

I would like to express my sincere gratitude to **Mr. Govind Bhokare** and **The Skybrisk Team** for providing this internship opportunity and their continuous guidance throughout the program.

This project — **Smart Bookstore Management System** — was assigned as Month 2 of my internship at The Skybrisk. The self-paced, structured program allowed me to explore full-stack development deeply and gain hands-on experience building a production-ready MERN application.

The project pushed me to implement real-world features including Stripe payment integration, Nodemailer email notifications, role-based access control, and cloud deployment on Vercel and Render — all of which significantly strengthened my technical skills.

I am grateful for the week-wise task structure, the encouragement to follow industry-standard practices, and the guidance on professional code organisation.

Thank you, Skybrisk Team, for making this a productive and rewarding learning experience.

Thank you, Skybrisk Team, for making this internship a productive and transformative experience.

Abstract

This report presents the design and development of a full-stack **Smart Bookstore Management System** built during Month 2 of the internship at **The Skybrisk**. The system is developed using the **MERN stack** (MongoDB, Express.js, React.js, Node.js) and provides a complete online bookstore experience including book browsing with search and genre filter, shopping cart, Stripe payment gateway, automated order confirmation emails, order tracking, and a comprehensive admin dashboard for managing books, orders, and users.

The backend follows RESTful API design with MVC architecture, JWT authentication, role-based access control, and Stripe webhook integration. The frontend uses React.js, Material-UI v5, Redux Toolkit, and React Hook Form with Yup validation. Both optional features from the internship specification — Stripe payment gateway and email notifications — were successfully implemented. The system is fully deployed on Vercel (frontend), Render (backend), and MongoDB Atlas (database) with automatic CI/CD via GitHub.

Contents

Acknowledgement **i**

Abstract **ii**

1 Introduction **1**

1.1 Overview of the Project 1
1.2 Internship Context 1
1.3 Objectives of the Project 2
1.4 Scope and Organisation 2

2 Literature Review **3**

2.1 Overview of Existing Bookstore Platforms 3
2.2 Traditional Bookstore vs Online System 3
2.3 Technologies Used 4

3 System Requirements and Analysis **5**

3.1 Problem Statement 5
3.2 Functional Requirements 5
3.3 Non-Functional Requirements 5
3.4 User Roles and Permissions 6
3.5 Project Timeline (Internship Specification) 6

4 System Architecture and UML Design **7**

4.1 Overall Architecture 7
4.2 Use Case Diagram 8
4.3 Entity-Relationship Diagram 9
4.4 Class Diagram 10
4.5 Activity Diagram 11
4.6 Sequence Diagram 12
4.7 Component Diagram 13
4.8 State Diagram 14
4.9 Object Diagram 15

4.10 Deployment Diagram 15

5 Database Design **16**

5.1 MongoDB Collections 16

5.1.1 Users 16

5.1.2 Books 16

5.1.3 Orders 16

5.1.4 OrderItems 16

5.1.5 Payments 16

5.1.6 EmailLogs 16

5.2 Entity Relationships 17

6 Backend Implementation **18**

6.1 Project Structure 18

6.2 JWT Authentication Flow 18

6.3 Stripe Payment Flow 18

6.4 REST API Endpoints 19

7 Frontend Implementation **20**

7.1 Project Structure 20

7.2 Redux Slices 20

7.3 Pages Built 21

8 API Testing **22**

8.1 Testing with Postman 22

8.2 Test Results 23

9 Deployment **24**

9.1 Architecture 24

9.2 Environment Variables 24

10 Challenges and Solutions **26**

10.1 Stripe Webhook Raw Body 26

10.2 CORS Policy on Production 26

10.3 Gmail App Password 26

10.4 Render Root Directory 26

11 Results and Outcomes **27**

11.1 Module Completion 27

11.2 Performance 28

Conclusion **29**

References **30**

A List of Figures **31**

B List of Tables **32**

C Glossary **33**

List of Figures

4.1 Use Case Diagram — Customer and Admin Interactions 8
4.2 Entity-Relationship Diagram — MongoDB Collections and Relationships . 9
4.3 Class Diagram — Mongoose Models with Attributes and Methods 10
4.4 Activity Diagram — Complete Customer Purchase Flow 11
4.5 Sequence Diagram — Order Placement and Stripe Payment Confirmation . 12
4.6 Component Diagram — Frontend Components and Backend Services . . . 13
4.7 State Diagram — Order and Payment Status Lifecycle 14
4.8 Object Diagram — Runtime Instance of a Completed Order 15
4.9 Deployment Diagram — Cloud Infrastructure on Vercel, Render, and MongoDB Atlas 15

List of Tables

2.1 Traditional Bookstore vs Online Bookstore System 3

3.1 User Roles and Permissions 6

3.2 Week-wise Development Timeline 6

5.1 Entity Relationships between Collections 17

6.1 Primary REST API Endpoints 19

7.1 Frontend Pages and Features 21

8.1 API Test Results Summary 23

9.1 Environment Variables 24

11.1 Module Completion Status 27

Chapter 1

Introduction

This project was assigned as Month 2 of the internship at **The Skybrisk**. The client is a bookstore owner who requires a complete web-based solution to manage books, customers, admins, and orders with online payment processing.

1.1 Overview of the Project

The Smart Bookstore Management System is a web-based platform that digitises the complete book sales lifecycle — from browsing and cart management to Stripe payment processing, email confirmation, order tracking, and administrative oversight. It serves two user types: customers who browse and purchase books, and administrators who manage inventory and orders.

1.2 Internship Context

This project was developed as the Month 2 assignment at The Skybrisk internship program. The specification required:

- Full-stack web application using React and Node.js with Express.js.
- JWT authentication with role-based access control (Customer and Admin).
- Book browsing with search and filter by genre, author, and title.
- Shopping cart and checkout with online payment.
- Admin dashboard for book and order management.
- MongoDB with proper schema design and Mongoose validation.
- Optional (implemented): Stripe payment gateway and email notifications.

1.3 Objectives of the Project

- Build a full-stack MERN bookstore with customer and admin interfaces.
- Implement secure JWT authentication with bcrypt password hashing.
- Develop book management with search, genre filter, and pagination.
- Integrate Stripe PaymentIntent API with webhook-based payment confirmation.
- Automate order confirmation emails using Nodemailer and Gmail SMTP.
- Log all email communications to MongoDB for audit visibility.
- Deploy the full system on cloud platforms with CI/CD via GitHub.

1.4 Scope and Organisation

The scope covers a complete MERN bookstore for customers and admins. This report has 12 chapters: Introduction, Literature Review, Requirements, Architecture and UML, Database Design, Backend, Frontend, API Testing, Deployment, Challenges, Results, and Conclusion.

Chapter 2

Literature Review

2.1 Overview of Existing Bookstore Platforms

Commercial platforms like Amazon Books and Flipkart are closed-source and unsuitable for custom deployment. The Skybrisk internship specification required building a custom solution from scratch, providing hands-on experience with every layer of a production full-stack application including payment processing and email automation.

2.2 Traditional Bookstore vs Online System

Table 2.1: Traditional Bookstore vs Online Bookstore System

Aspect	Traditional	Online System
Inventory	Manual spreadsheets	MongoDB CRUD with admin UI
Customer Reach	Local, limited hours	Global, 24/7 access
Payment	Cash only	Stripe payment gateway
Order Tracking	Manual phone calls	Real-time status updates
Communication	Manual	Automated email confirmations
Reporting	None	Admin dashboard, revenue stats

2.3 Technologies Used

The MERN stack is the industry choice for e-commerce development. MongoDB provides flexible document storage. Express.js handles RESTful routing. React.js with Redux Toolkit provides predictable state management for cart and payment flows. Stripe is the industry-standard payment gateway with webhook support. Nodemailer with Gmail SMTP provides reliable transactional email.

Chapter 3

System Requirements and Analysis

3.1 Problem Statement

As specified in the Month 2 internship assignment at The Skybrisk: a bookstore owner requires a complete web-based solution to manage books, users (customers and admins), and orders. The system must support online book browsing, cart management, order placement, Stripe payment, and admin inventory management.

3.2 Functional Requirements

- Users must register and login securely with JWT authentication.
- Customers must browse books with genre filter, search, and pagination.
- Customers must manage a Redux cart and proceed to Stripe checkout.
- Successful payment must update order status, send email, and log to MongoDB.
- Admins must perform full CRUD on books with search and stock management.
- Admins must view all orders and update statuses per order.
- Admin dashboard must display stats: total books, orders, paid revenue, recent orders.

3.3 Non-Functional Requirements

- Responsive UI using Material-UI v5 across all device sizes.
- Secure password storage with bcrypt (salt rounds: 10).
- Modular MVC backend and feature-based Redux frontend.
- CORS configured for trusted origins only.
- Environment-based configuration for local and production.

3.4 User Roles and Permissions

Table 3.1: User Roles and Permissions

Role	Module Access	Permissions
Customer	Books, Cart, Checkout, My Orders	Browse, add to cart, pay, view own orders
Admin	All modules + Dashboard	Full CRUD on books, view/update all orders, email logs

3.5 Project Timeline (Internship Specification)

Table 3.2: Week-wise Development Timeline

Week	Tasks
1	UML diagrams (10 types), project setup, backend structure, JWT auth
2	Book management CRUD, frontend views, search, filter, pagination
3	Order processing, Stripe integration, cart/checkout, admin dashboard, email
4	Testing, UI polish, Postman documentation, cloud deployment

Chapter 4

System Architecture and UML Design

4.1 Overall Architecture

The system follows a three-tier architecture:

- **Presentation Layer:** React.js + Redux Toolkit + Material-UI v5 on Vercel.
- **Application Layer:** Node.js + Express.js REST API on Render.
- **Data Layer:** MongoDB Atlas + Mongoose ODM with 6 collections.

External services: Stripe API (PaymentIntent + webhook) and Gmail SMTP (Nodemailer).

4.2 Use Case Diagram

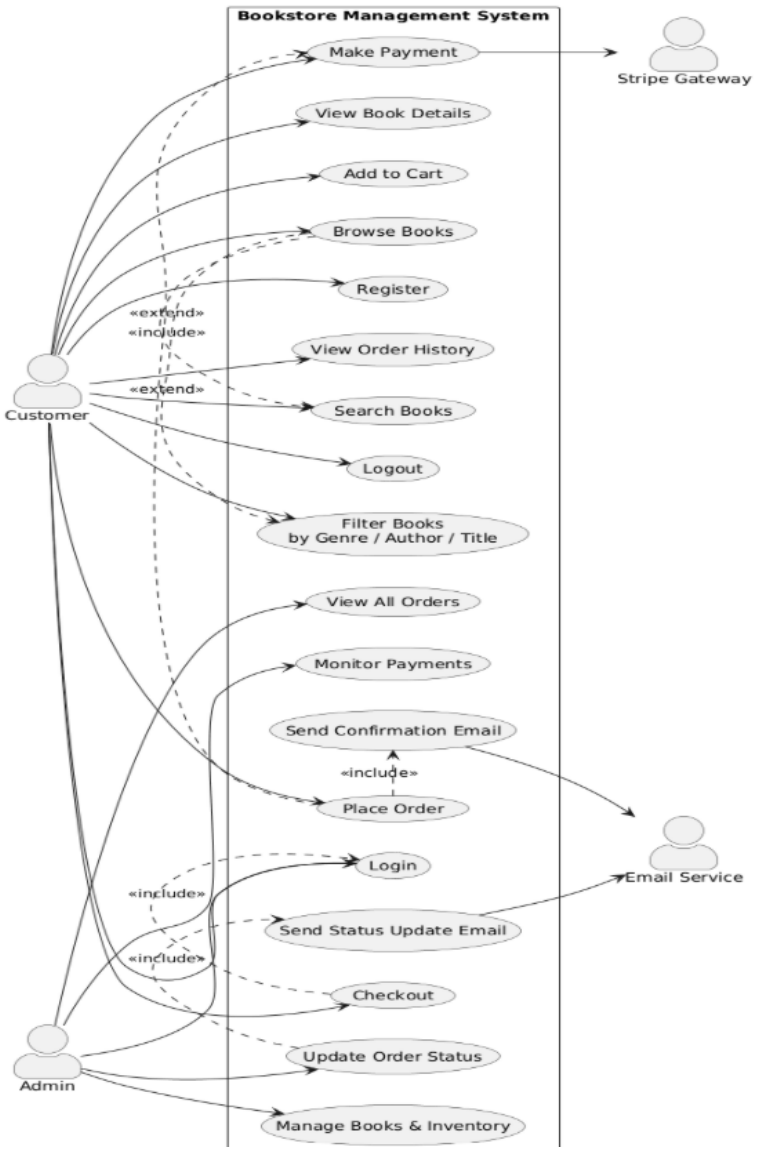

Figure 4.1: Use Case Diagram — Customer and Admin Interactions

Customer use cases: Register, Login, Browse Books, Search/Filter, View Details, Add to Cart, Checkout, Make Payment, View Orders, Logout. Admin use cases: Manage Books (CRUD), View/Update All Orders, View Dashboard, View Email Logs.

4.3 Entity-Relationship Diagram

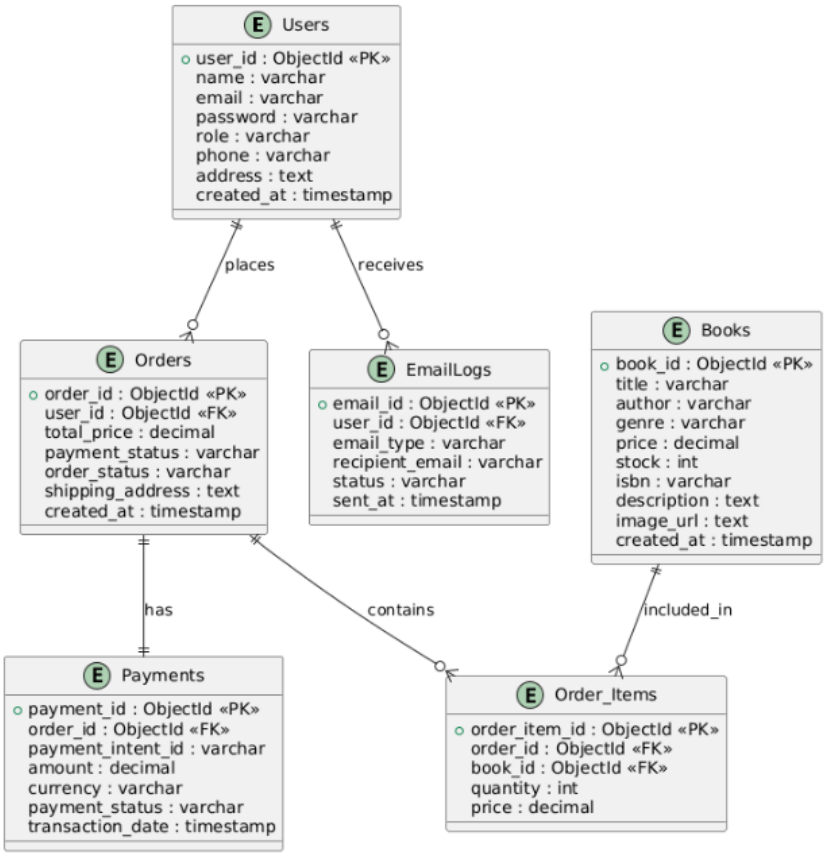

Figure 4.2: Entity-Relationship Diagram — MongoDB Collections and Relationships

Six collections: Users, Books, Orders, OrderItems, Payments, EmailLogs. ObjectId references used throughout. Mongoose `populate()` resolves references like SQL JOINs.

4.4 Class Diagram

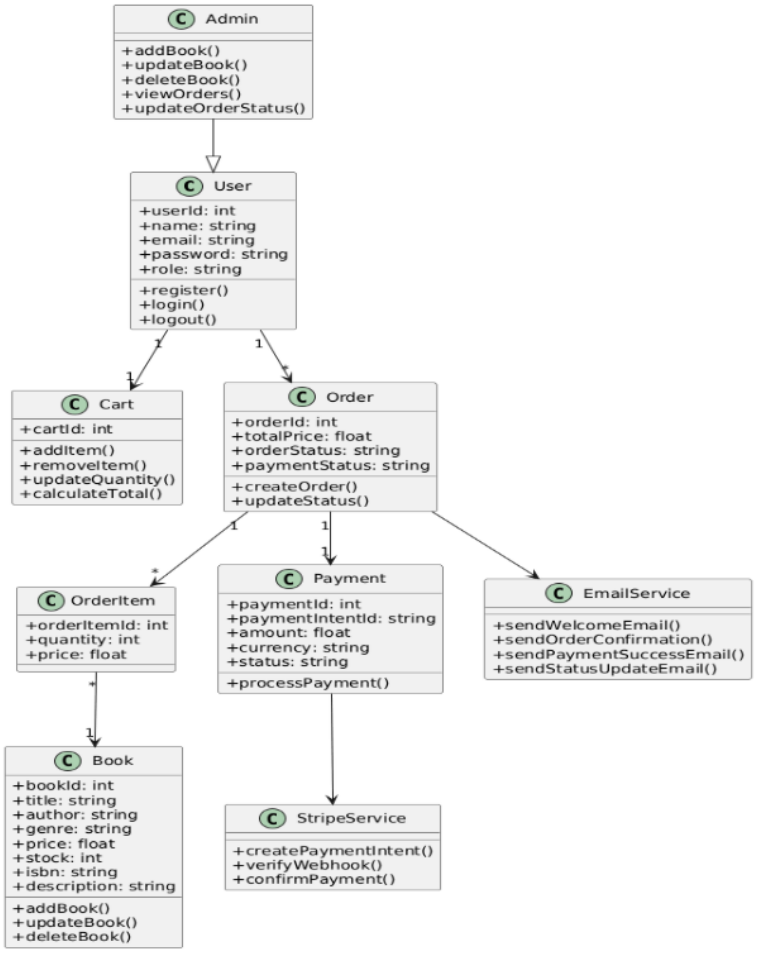

Figure 4.3: Class Diagram — Mongoose Models with Attributes and Methods

4.5 Activity Diagram

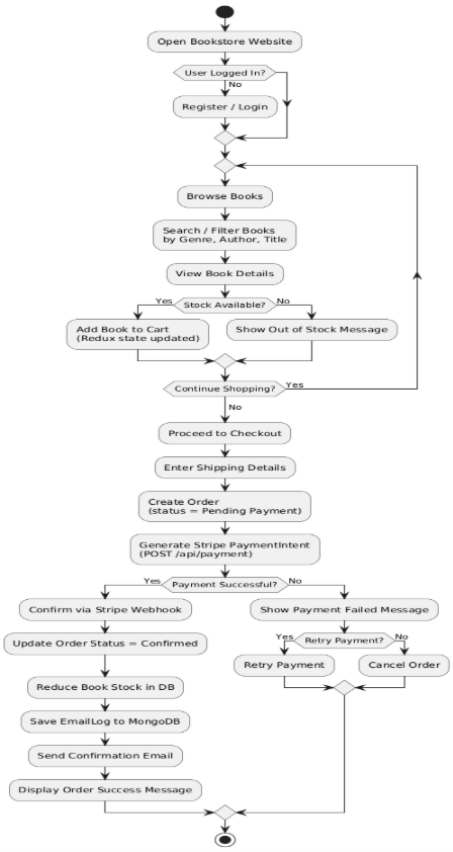

Figure 4.4: Activity Diagram — Complete Customer Purchase Flow

4.6 Sequence Diagram

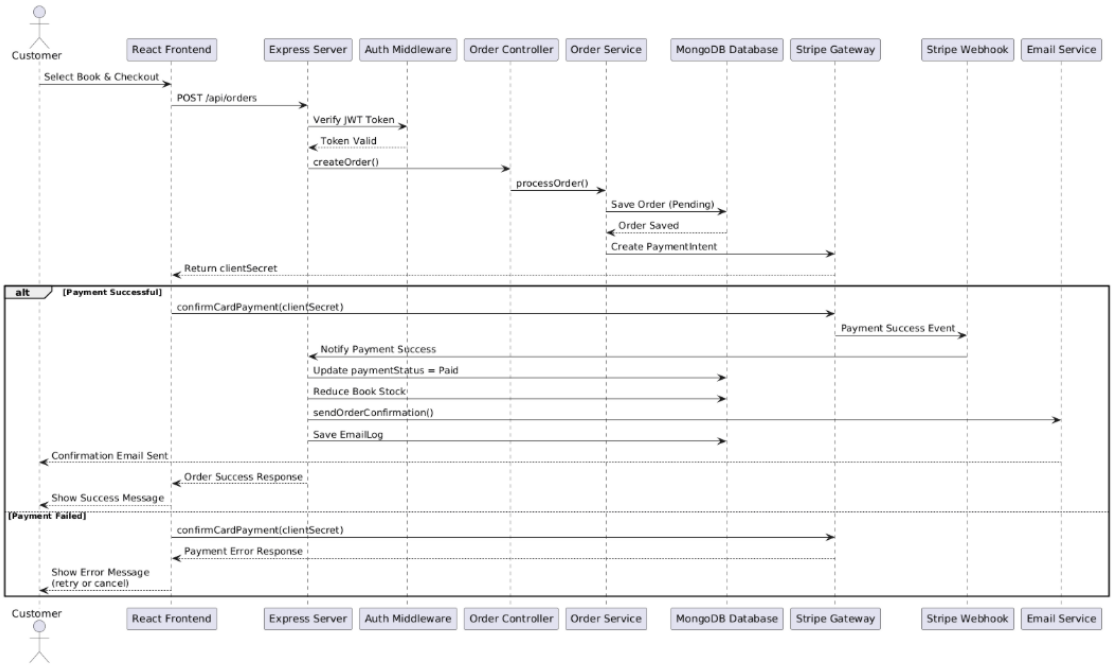

Figure 4.5: Sequence Diagram — Order Placement and Stripe Payment Confirmation

4.7 Component Diagram

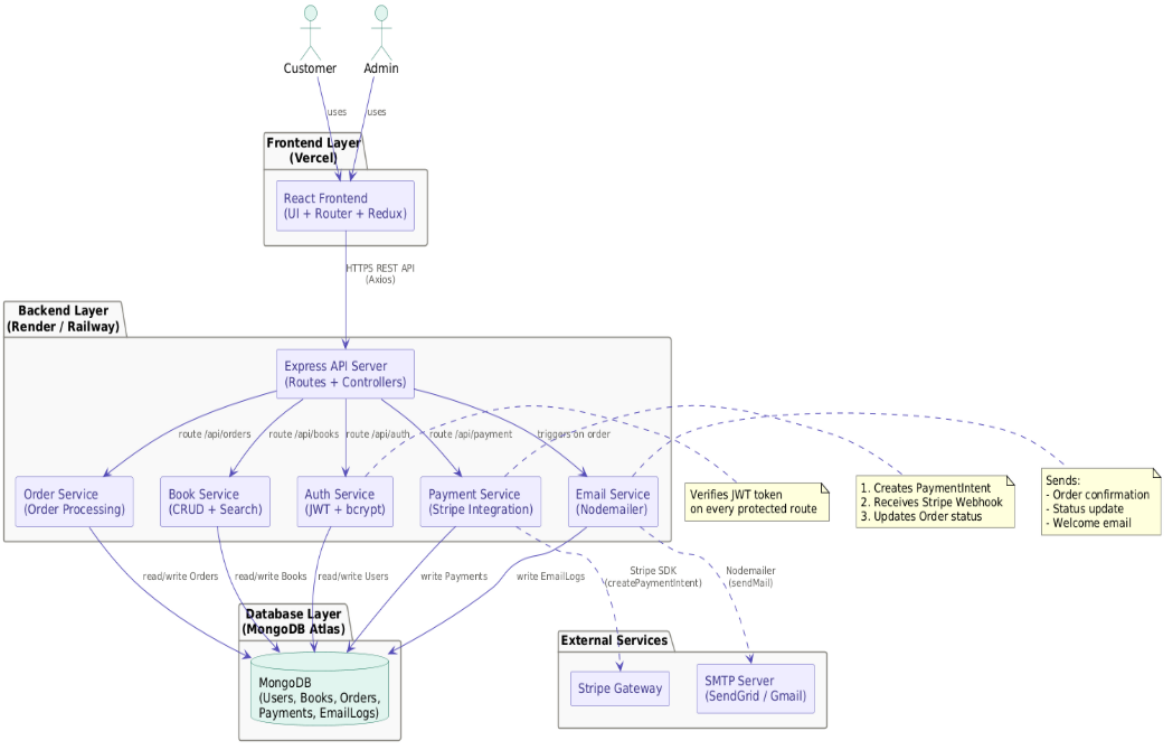

Figure 4.6: Component Diagram — Frontend Components and Backend Services

4.8 State Diagram

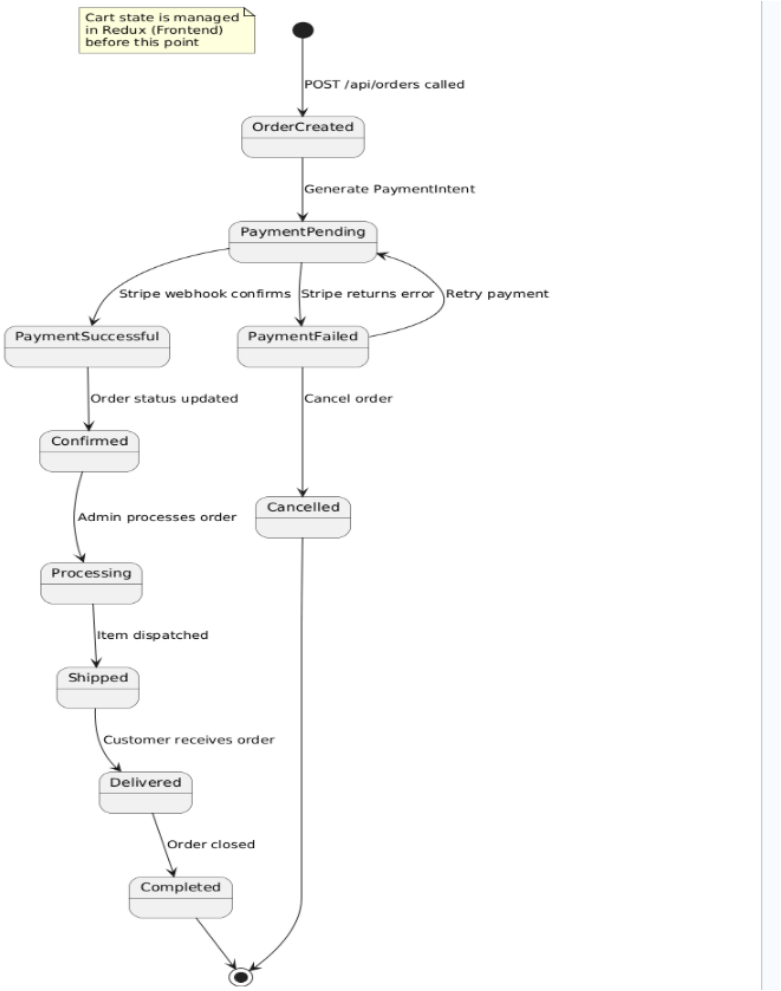

Figure 4.7: State Diagram — Order and Payment Status Lifecycle

4.9 Object Diagram

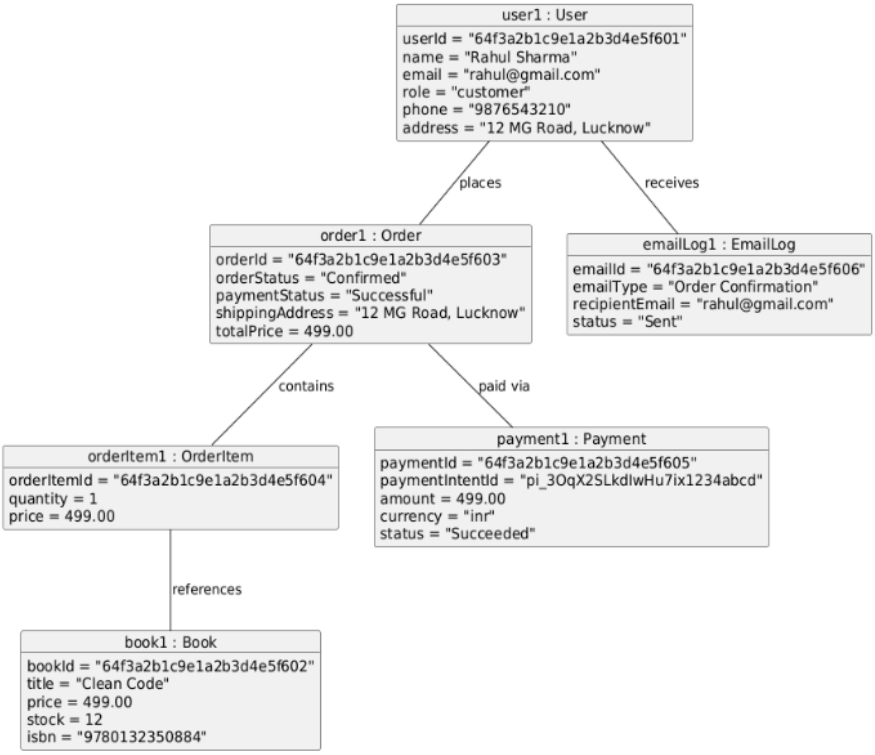

Figure 4.8: Object Diagram — Runtime Instance of a Completed Order

4.10 Deployment Diagram

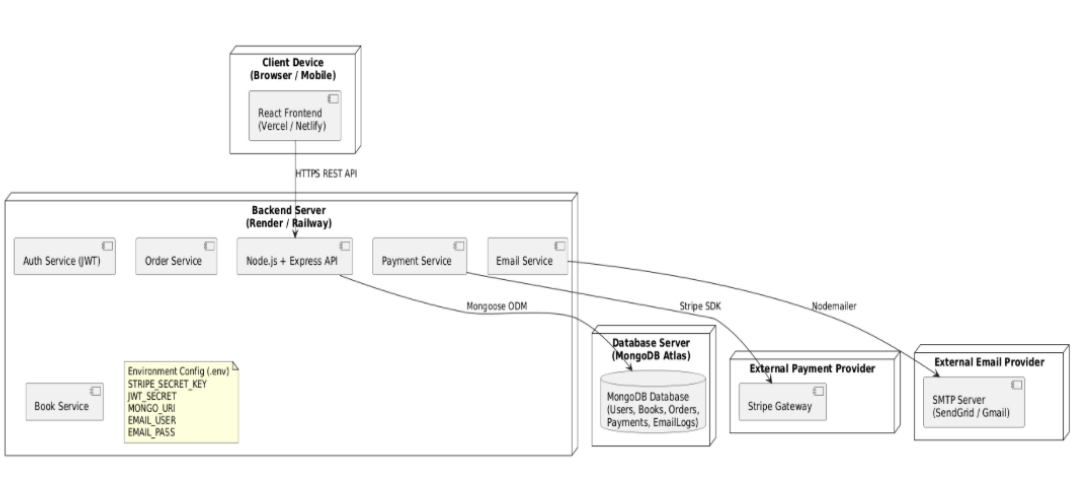

Figure 4.9: Deployment Diagram — Cloud Infrastructure on Vercel, Render, and MongoDB Atlas

Chapter 5

Database Design

5.1 MongoDB Collections

5.1.1 Users

Fields: `name`, `email` (unique), `password` (bcrypt hash), `role` (customer/admin), `phone`, `address`. Role defaults to `customer`. Admin assigned manually via MongoDB Atlas.

5.1.2 Books

Fields: `title`, `author`, `genre`, `price`, `stock`, `isbn` (unique), `description`, `imageUrl`. Stock decremented on order creation.

5.1.3 Orders

Fields: `userId` (ref: Users), `totalPrice`, `orderStatus` (enum: Created/Confirmed/Processing/Shipped/Delivered/Cancelled), `paymentStatus` (Pending/Paid/Failed), `shippingAddress`.

5.1.4 OrderItems

Fields: `orderId` (ref: Orders), `bookId` (ref: Books), `quantity`, `price` (at time of purchase, preserved for historical accuracy).

5.1.5 Payments

Fields: `orderId` (ref: Orders), `paymentIntentId` (Stripe), `amount`, `currency` (inr), `status` (Pending/Succeeded/Failed).

5.1.6 EmailLogs

Fields: `userId` (ref: Users), `emailType`, `recipientEmail`, `status` (Sent/Failed), `sentAt`. Created for every email attempt regardless of outcome.

5.2 Entity Relationships

Table 5.1: Entity Relationships between Collections

From	To	Type	Field
Orders	Users	Many-to-One	userId (ObjectId)
OrderItems	Orders	Many-to-One	orderId (ObjectId)
OrderItems	Books	Many-to-One	bookId (ObjectId)
Payments	Orders	One-to-One	orderId (ObjectId)
EmailLogs	Users	Many-to-One	userId (ObjectId)

Chapter 6

Backend Implementation

6.1 Project Structure

```
bookstore-backend/
          config/        (db.js, stripe.js, email.js)
          controllers/   (auth, book, order, payment, email)
          middleware/    (authMiddleware, adminMiddleware,
   errorMiddleware)
          models/        (User, Book, Order, OrderItem, Payment,
   EmailLog)
          routes/        (5 route files)
          services/      (emailService, stripeService)
          utils/         (generateToken, validators)
          server.js
```

6.2 JWT Authentication Flow

1. Password verified against bcrypt hash using `bcrypt.compare()`.
2. JWT signed with `jwt.sign()` containing `_id`, 30-day expiry.
3. Token stored in `localStorage`, sent via Axios interceptor as Bearer header.
4. `authMiddleware` verifies token, attaches user to `req.user`.
5. `adminMiddleware` checks `req.user.role === 'admin'`, returns 403 if not.

6.3 Stripe Payment Flow

1. `POST /api/orders` creates order with `paymentStatus: Pending`.
2. `POST /api/payment/create-intent` creates Stripe PaymentIntent with metadata, returns `clientSecret`.

3. Frontend calls `stripe.confirmCardPayment(clientSecret)`.
4. Stripe fires `payment_intent.succeeded` to `POST /api/payment/webhook`.
5. Webhook updates order to Confirmed/Paid and sends confirmation email.

6.4 REST API Endpoints

Table 6.1: Primary REST API Endpoints

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register new user
POST	/api/auth/login	Public	Login, returns JWT
GET	/api/auth/profile	Private	Get own profile
GET	/api/books	Public	List books (search/filter)
POST	/api/books	Admin	Create new book
PUT	/api/books/:id	Admin	Update book
DELETE	/api/books/:id	Admin	Delete book
POST	/api/orders	Private	Place new order
GET	/api/orders/my	Private	Get own orders
GET	/api/orders	Admin	Get all orders
PUT	/api/orders/:id/status	Admin	Update order status
POST	/api/payment/create-intent	Private	Create Stripe PaymentIntent
POST	/api/payment/webhook	Public	Stripe webhook handler
GET	/api/emails	Admin	Get email logs

Chapter 7

Frontend Implementation

7.1 Project Structure

```
bookstore-frontend/src/
          api/          (axiosInstance, authApi, bookApi,
                    orderApi, paymentApi, emailApi)
          app/          (store.js)
          components/   (Navbar, Footer, BookCard, BookFilter,
                    BookSearch, CartItem, CheckoutForm)
          features/     (auth, books, cart, orders, payment slices
   )
          pages/        (12 pages for customer and admin flows)
          utils/
```

7.2 Redux Slices

Five slices manage global state: **authSlice** (JWT, user), **bookSlice** (list, pagination), **cartSlice** (frontend-only, no backend persistence), **orderSlice** (creation, history, admin), **paymentSlice** (clientSecret).

7.3 Pages Built

Table 7.1: Frontend Pages and Features

Page	Features
LoginPage	React Hook Form + Yup, JWT on login
RegisterPage	Form validation, auto-login
BooksListPage	CSS Grid, BookFilter sidebar, BookSearch, pagination
BookDetailPage	2-column, sticky image, Add to Cart
CartPage	Quantity control, summary, Stripe badge
CheckoutPage	2-step: shipping address + Stripe CardElement
MyOrdersPage	Order cards, status chips
OrderSuccessPage	Confirmation, email sent message
AdminDashboard	Stats cards, revenue, recent orders
AdminBooksPage	Full CRUD, dialog form, stock chips
AdminOrdersPage	All orders, status update dropdown

Chapter 8

API Testing

8.1 Testing with Postman

All 17 endpoints tested using Postman with environment variables for base URL and JWT tokens. Stripe CLI used for local webhook: `stripe listen --forward-to localhost:5000/api/paym`

8.2 Test Results

Table 8.1: API Test Results Summary

Test Scenario	Expected Result	Status
POST /api/auth/register	201, JWT returned	PASS
POST /api/auth/login valid	200, JWT returned	PASS
GET /api/auth/profile no token	401 Unauthorized	PASS
POST /api/books customer token	403 Forbidden	PASS
POST /api/books admin token	201 Created	PASS
GET /api/books with search	200, filtered	PASS
POST /api/orders valid IDs	201 Created	PASS
GET /api/orders/my	200, list	PASS
GET /api/orders admin	200, all orders	PASS
PUT order status admin	200, updated	PASS
POST payment create-intent	200, clientSecret	PASS
POST payment webhook CLI	200, email sent	PASS
GET /api/emails admin	200, email logs	PASS
Payment with 4242 test card	Order confirmed + email	PASS

Chapter 9

Deployment

9.1 Architecture

- **Frontend:** Vercel — `https://bookstorefrontend-m2vv.vercel.app`
- **Backend:** Render — `https://bookstore-backend-l20s.onrender.com`
- **Database:** MongoDB Atlas (Singapore cluster)
- **Payments:** Stripe webhook on Render URL (Active)
- **Email:** Gmail SMTP via Nodemailer

9.2 Environment Variables

Table 9.1: Environment Variables

Variable	Description	Platform
`MONGO_URI`	MongoDB Atlas connection	Render
`JWT_SECRET`	JWT signing key	Render
`STRIPE_SECRET_KEY`	Stripe secret API key	Render
`STRIPE_WEBHOOK_SECRET`	Webhook signing secret	Render
`EMAIL_USER`	Gmail address	Render
`EMAIL_PASS`	Gmail App Password	Render
`NODE_ENV`	production	Render
`VITE_API_URL`	Render backend URL	Vercel
`VITE_STRIPE_PUBLIC_KEY`	Stripe public key	Vercel

CI/CD: Every push to `main` branch triggers automatic redeploy on both Vercel and Render.

Chapter 10

Challenges and Solutions

10.1 Stripe Webhook Raw Body

Challenge: `express.json()` parses body before webhook can verify Stripe signature. **Solution:** Register webhook route with `express.raw()` before `express.json()` in `server.js`.

10.2 CORS Policy on Production

Challenge: Vercel frontend blocked by CORS after deployment. **Solution:** Added Vercel URL to CORS `origin` array in `server.js` and redeployed.

10.3 Gmail App Password

Challenge: Nodemailer rejected regular Gmail password. **Solution:** Generated 16-character App Password from Google Account Security settings.

10.4 Render Root Directory

Challenge: Render could not find `package.json` in monorepo. **Solution:** Set Root Directory to `bookstore-backend` in Render service settings.

Chapter 11

Results and Outcomes

11.1 Module Completion

Table 11.1: Module Completion Status

Module	Features	Status
Authentication	JWT login, registration, role-based	Done
Book Browsing	Genre filter, search, grid layout	Done
Shopping Cart	Add/remove/update, summary	Done
Stripe Checkout	Address + card payment (2-step)	Done
Webhook Payment	Confirms order via Stripe event	Done
Email Notification	Gmail SMTP on payment confirmed	Done
Email Logging	EmailLog MongoDB collection	Done
Admin Dashboard	Stats, revenue, orders	Done
Admin Books CRUD	Add/edit/delete dialog	Done
Admin Orders	Status update per row	Done
Cloud Deployment	Vercel + Render + Atlas	Done
Optional: Stripe	Fully implemented	Done
Optional: Email	Fully implemented	Done

11.2 Performance

- API response time: under 200 ms on Render free tier.
- Frontend load: under 2 seconds on Vercel CDN.
- Stripe payment end-to-end: under 5 seconds with test card.
- Email delivery: under 10 seconds after webhook confirmation.

Conclusion

Summary

This project involved the complete design, development, testing, and deployment of a full-stack Smart Bookstore Management System as Month 2 of the internship at The Skybrisk. All required modules from the project specification were successfully implemented, along with both optional features — Stripe payment gateway and email notifications — which were integrated as production-ready webhook-confirmed payments and automated transactional email with MongoDB audit logging.

Technical Skills Developed

- RESTful API development with JWT authentication and Stripe webhook handling.
- MongoDB database design with six collections and Mongoose ODM.
- React.js with Redux Toolkit, Material-UI v5, and React Hook Form.
- Stripe PaymentIntent API with production webhook signature verification.
- Nodemailer with Gmail SMTP for transactional email and audit logging.
- Cloud deployment with CI/CD via GitHub on Vercel and Render.

Internship Learning

This Month 2 project at The Skybrisk provided deep, practical experience bridging academic knowledge with professional software engineering. Building a production-ready application with real payment processing, cloud deployment, and automated email has significantly strengthened readiness for industry roles in full-stack development. The structured Skybrisk internship program was instrumental in guiding this learning journey.

References

[1] MongoDB Documentation. https://www.mongodb.com/docs

[2] Express.js Documentation. https://expressjs.com

[3] React.js Documentation. https://react.dev

[4] Node.js Documentation. https://nodejs.org/en/docs

[5] Mongoose Documentation. https://mongoosejs.com/docs

[6] JSON Web Token. https://jwt.io

[7] Material-UI Documentation. https://mui.com

[8] Redux Toolkit. https://redux-toolkit.js.org

[9] Stripe API Documentation. https://stripe.com/docs

[10] Stripe Webhooks Guide. https://stripe.com/docs/webhooks

[11] Nodemailer Documentation. https://nodemailer.com

[12] React Hook Form. https://react-hook-form.com

[13] Yup Validation. https://github.com/jquense/yup

[14] Vercel Documentation. https://vercel.com/docs

[15] Render Documentation. https://render.com/docs

[16] MongoDB Atlas. https://www.mongodb.com/atlas

[17] bcrypt.js. https://github.com/dcodeIO/bcrypt.js

[18] Axios. https://axios-http.com/docs/intro

[19] React Router v6. https://reactrouter.com/en/main

[20] Stripe CLI. https://stripe.com/docs/stripe-cli

Appendix A

List of Figures

Figure 1: Use Case Diagram

Figure 2: Entity-Relationship Diagram

Figure 3: Class Diagram

Figure 4: Activity Diagram

Figure 5: Sequence Diagram

Figure 6: Component Diagram

Figure 7: State Diagram

Figure 8: Object Diagram

Figure 9: Deployment Diagram

Appendix B

List of Tables

Table 1: Traditional vs Online Bookstore

Table 2: User Roles and Permissions

Table 3: Week-wise Timeline

Table 4: Entity Relationships

Table 5: REST API Endpoints

Table 6: Frontend Pages

Table 7: API Test Results

Table 8: Environment Variables

Table 9: Module Completion Status

Appendix C

Glossary

MERN	MongoDB, Express.js, React.js, Node.js full-stack framework.
JWT	JSON Web Token for stateless authentication.
REST	Representational State Transfer API architecture.
CRUD	Create, Read, Update, Delete operations.
MVC	Model-View-Controller design pattern.
ODM	Object Document Mapper (Mongoose for MongoDB).
Webhook	HTTP callback triggered by Stripe on payment success.
bcrypt	Password hashing with configurable salt rounds.
CI/CD	Continuous Integration/Deployment via GitHub auto-deploy.
CDN	Content Delivery Network for fast global asset delivery.